

MultiExplorer - How to extend the tool

Erick R. Amorim, Gregório K. Neto, Igor O. Mello, Liana D. Duenha,
Samuel S. Rodrigues, Laura B. Ludgero, Gislayne G. Damasceno,
Ricardo R. dos Santos

September 10, 2024

1 How to use GPGPU-Sim to simulate gpu models

This section describes how to simulate a GPU model with GPGPU-Sim.

1.1 Download CUDA version 11.0 or earlier.

```
https://developer.nvidia.com/cuda-11.0-download-archive
```

1.2 Clone GPGPU-Sim repository

```
git clone https://github.com/gpgpu-sim/gpgpu-sim_distribution.git
```

The GPGPU-Sim repository can also be found in the MultiExplorer folder.”

```
/multiexplorer/gpgpu-sim_distribution
```

If the folder is empty, run the following command:

```
git submodule update --init --recursive
```

1.3 Install dependencies

```
sudo apt-get install build-essential xutils-dev bison  
↳ zlib1g-dev flex libglu1-mesa-dev
```

```
sudo apt-get install doxygen graphviz
```

```
sudo apt-get install python-pmw python-ply python-numpy  
↳ libpng12-dev python-matplotlib
```

```
sudo apt-get install libxi-dev libxmu-dev libglut3-dev
```

1.4 Build GPGPU-Sim

In the GPGPU-Sim directory ('gpgpu-sim_distribution'), execute:

```
export CUDA\_INSTALL\_PATH=/usr/local/cuda-11.0/
(The CUDA location must be updated if it differs)

source setup_environment

make
```

1.5 Executing

Create an execution folder. In this folder, place the GPU gpgpusim.config, XML file, and config_fermi_islip.icnt (if present). Additionally, put the application executable in the same folder. Export the CUDA_INSTALL_PATH in the folder.

```
export CUDA_INSTALL_PATH=/usr/local/cuda-11.0/
```

Execute the setup_environment from the 'gpgpu-sim_distribution' in your new configuration folder.

```
source {setup_environment_location}
```

Example:

```
source /home/lscad/Desktop/gpgpu-sim_distribution/
↪ setup_environment
```

After setting up the environment, execute the application:

```
./{application_name}
```

The simulation will start running in the terminal. To save the output to a TXT file, execute:

```
./{application_name} > output.txt
```

1.6 Used applications and pre-simulated results

The applications used can be found at:

```
/multiexplorer/Benchmarks.zip
```

The pre-simulated results can be found at:

```
/multiexplorer/MultiExplorer/src/MultiexplorerGPGPU/
↪ DS_DSE/db/Experimentos/
```

1.7 Problems

If a PTXAS problem occurs, use a PTXAS executable from another CUDA version. Replace the PTXAS executable in the CUDA folder with the one located at:

```
https://drive.google.com/drive/folders/1
↪ Gm3Wdrcrengl0tIC9BqAED9oBSPk7Yz7?usp=sharing
```

2 How to expand MultiExplorer for new execution flows

This section provides a high-level overview of how to integrate new execution flows into the MultiExplorer tool. The complexity and specific steps may vary depending on the flow and the features the user intends to incorporate.

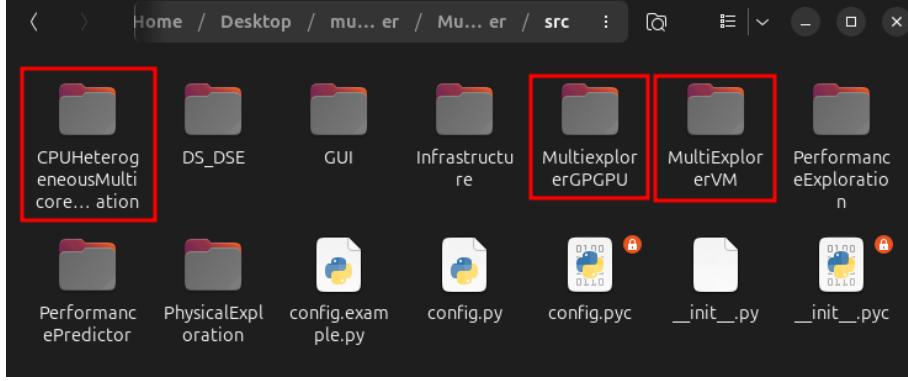


Figure 1: MultiExplorer source code folder.

To integrate an execution flow, a folder representing this flow must be created in the MultiExplorer source directory: `/multiexplorer/MultiExplorer/src/`. Figure 1 shows the currently available execution flows.

Within this new execution flow folder, five Python files can be created, along with any necessary dependency files. These files include: **AllowedValues.py**, **Steps.py**, **Adapters.py**, **Presenters.py**, and **MultiExplorer{flow_name}.py**.

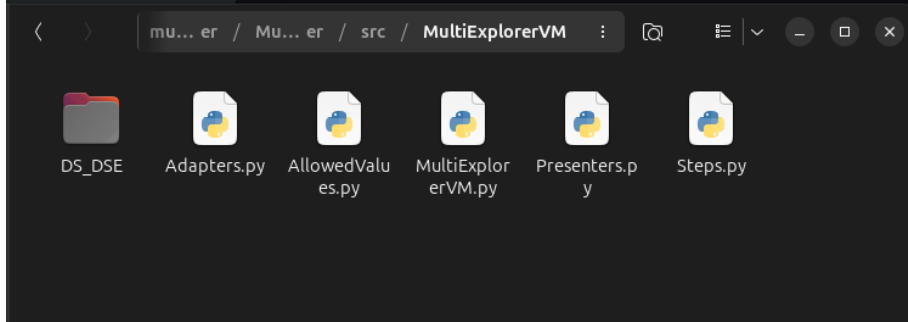


Figure 2: Execution flow folder.

Figure 2 illustrates an example of an execution flow folder structure.

2.1 Steps

Each execution flow is divided into steps, with each step performing a different type of processing. For the CPU flow, there are three steps: simulation, physical exploration, and DSE. For the GPU and VM flows, there are two steps: simulation and DSE. The difference is that physical exploration is not necessary for the VM flow and is integrated into the simulation tool for GPUs.

Each step will appear as a tab in the MultiExplorer interface - if the step requires user inputs - as shown in Figure 3.

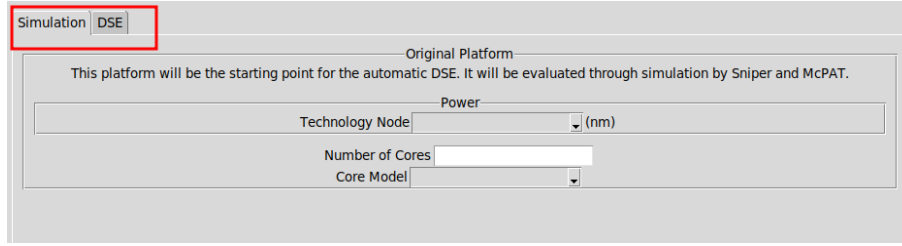


Figure 3: A execution flow tabs.

The python code (Code 1) provides an example of a **Steps.py** file with a step class.

The **Steps.py** file contains functions to handle its adapters results (`get_results`), user inputs (`get_user_inputs`), the step presenter (`get_presenter`), and the step name to be displayed in the interface (`get_label`).

```

1 import os
2 import sys
3 from ..Infrastructure.Events import Event
4 from ..Infrastructure.ExecutionFlow import Step
5 from Presenters import CloudSimPresenter, NSGAPresenter
6 from Adapters import CloudsimAdapter, NsgaIIPredDSEAdapter
7
8 ...
9
10 class NSGAIIDSEStep(Step):
11
12     def get_results(self):
13         return self.adapter.get_results()
14
15     def get_presenter(self):
16         return NSGAPresenter()
17
18     def __new__(cls):
19         if not hasattr(cls, 'instance'):
20             cls.instance = super(
21                 NSGAIIDSEStep,
22                 cls
23             ).__new__(cls)
24
25         return cls.instance
26
27     def __init__(self):
28         super(NSGAIIDSEStep, self).__init__()
29
30         self.adapter = NsgaIIPredDSEAdapter()
31
32     @staticmethod
33     def get_label():

```

```

34         return 'DSE'
35
36     @staticmethod
37     def has_user_input():
38         return True
39
40     def get_user_inputs(self):
41         return self.adapter.get_user_inputs()
42
43     def __execute__(self):
44         self.execution_exception = None
45
46         try:
47             self.adapter.execute()
48         except BaseException as exception:
49             self.execution_exception = exception
50
51     def __finish__(self):
52         if self.execution_exception is None:
53             self.fire(Event.STEP_EXECUTION_ENDED)
54         else:
55             self.fire(Event.STEP_EXECUTION_FAILED, self)

```

Listing 1: Steps Python code

2.2 Adapters

The adapter is the file that handles a step's operations, such as obtaining user inputs, performing file operations, and executing applications. The functions in this file depend on the specific step.

In this **Adapters.py** example, the adapter handles user inputs for the specific step. Each user input has configurations such as type, default value, and min-max. The adapter includes functions that prepare JSON files according to these user inputs, such as 'prepare', 'change_json_parameters', etc. With these modifications and preparations, the respective DSE algorithms will be executed in 'execute', such as 'dse' or/and 'brute_force'. The adapter must also include the 'get_results' function so that data can be accessed by other files (e.g., Presenters).

Code 2 illustrates the corresponding adapter for the step shown in Code 1.

```

1
2 class NsgaIIPredDSEAdapter(Adapter):
3
4     def __init__(self):
5         Adapter.__init__(self)
6
7         self.set_inputs([
8             InputGroup({
9                 'label': "Exploration Space",
10                'key': 'exploration_space',
11                'inputs': [
12                    Input({

```

```

13         'label': 'Run brute force aswell',
14         'key': 'run_brute_force',
15         "is_user_input": True,
16         "required": False,
17         'type': InputType.Checkbutton,
18         'value': True,
19     }),
20     Input({
21         'label': 'Original number VM for
    ↪ design',
22         'key': 'original_vm_for_design',
23         "is_user_input": True,
24         "required": True,
25         'type': InputType.IntegerRange,
26         'min_val': 1,
27         'max_val': 31,
28     })
29 ]
30     }),
31     ...
32
33 def execute(self):
34     self.prepare()
35
36     if self.inputs['exploration_space']['run_brute_force'
    ↪ ]:
37         self.dse_brute_force()
38
39     self.dse()
40     self.register_nsga_results()
41     self.register_brute_force_results()
42     self.ord_values()
43
44 def get_json_name(self):
45     ...
46
47 def get_input_json(self):
48     return self.project_folder + '/' + self.get_json_name
    ↪ ()
49
50 def change_json_parameters(self):
51     ...
52
53 def get_json_data(self):
54     with open(self.input_json) as json_file:
55         self.json_data = json.load(json_file)
56
57 def prepare(self):
58     self.project_folder = self.get_output_path()
59     self.input_json = self.get_input_json()
60     self.change_json_parameters()

```

```

61         self.get_json_data()
62
63     def dse(self):
64         self.nsga = Nsga2Main (
65             self.project_folder,
66             self.input_json,
67             float(self.inputs['nsga_parameters'] [
68                 ↪ 'mutation_strength'])/100,
69             float(self.inputs['nsga_parameters'] [
70                 ↪ 'mutation_rate'])/100,
71             int(self.inputs['nsga_parameters'] ['
72                 ↪ num_of_individuals']),
73             int(self.inputs['nsga_parameters'] ['
74                 ↪ num_of_generations']),
75             self.inputs['exploration_space'] ['
76                 ↪ run_prediction']
77         )
78         print("DSE NSGA2: OK")
79
80     def dse_brute_force(self):
81         self.brute_force = DsDseBruteForce(self.
82             ↪ project_folder, self.input_json, self.inputs [
83             ↪ exploration_space'] ['run_prediction'])
84         print("DSE Brute Force: OK")
85
86     def register_nsga_results(self):
87         ...
88
89     def register_brute_force_results(self):
90         ...
91
92     def get_results(self):
93         return self.presentable_results
94
95     def ord_values(self):
96         ..
97
98     def get_filtered_results(self, solutions):
99         ...

```

Listing 2: Adapters Python code

2.3 AllowedValues

The **AllowedValues.py** file contains lists of data. If a user input needs to be selected from a predefined set of models, a class representing this list must be created in the **AllowedValues.py** file and used in the 'type' field within adapters. For example, in the GPU execution flow (Code 3), there is a list of nine applications for the *benchmark*. This approach allows for easy expansion of the application database. Additionally, it provides other benefits, such as simplifying the retrieval of a JSON file path for each model.

```

1
2 from enum import Enum
3 from ..config import PATH_INPUTS
4
5 ...
6
7 class Applications(Enum):
8     asyncAPI = 1
9     backprop = 2
10    bfs = 3
11    clock = 4
12    dwt2d = 5
13    hotspot = 6
14    needle = 7
15    nn = 8
16    vectorAdd = 9
17
18    @staticmethod
19    def belongs(value):
20        return value in set(item.value for item in
21                               ↪ Applications)
22
23    @staticmethod
24    def get_label(value):
25
26        if value == Applications.asyncAPI:
27            return "asyncAPI"
28        if value == Applications.backprop:
29            return "backprop"
30        if value == Applications.bfs:
31            return "bfs"
32        if value == Applications.clock:
33            return "clock"
34        if value == Applications.dwt2d:
35            return "dwt2d"
36        if value == Applications.hotspot:
37            return "hotspot"
38        if value == Applications.needle:
39            return "needle"
40        if value == Applications.nn:
41            return "nn"
42        if value == Applications.vectorAdd:
43            return "vectorAdd"
44
45        raise ValueError("Value does not corresponds to a
46                               ↪ known predicted core.")
47
48    @staticmethod
49    def get_dict():
50
51        return {

```

```

50     Applications.asyncAPI.value: Applications.
        ↳ get_label(Applications.asyncAPI),
51     Applications.backprop.value: Applications.
        ↳ get_label(Applications.backprop),
52     Applications.bfs.value: Applications.get_label(
        ↳ Applications.bfs),
53     Applications.clock.value: Applications.get_label(
        ↳ Applications.clock),
54     Applications.dwt2d.value: Applications.get_label(
        ↳ Applications.dwt2d),
55     Applications.hotspot.value: Applications.
        ↳ get_label(Applications.hotspot),
56     Applications.needle.value: Applications.get_label
        ↳ (Applications.needle),
57     Applications.nn.value: Applications.get_label(
        ↳ Applications.nn),
58     Applications.vectorAdd.value: Applications.
        ↳ get_label(Applications.vectorAdd),
59 }

```

Listing 3: AllowedValues Python code

2.4 Presenters

The **Presenters.py** is the file responsible for the presentation of the results in the form of tables, graphs, previews, and more. A class for each table and each graph - NSGA-II and Brute Force - must be created. Additionally, a class for the previews information in the processing window.

The datas for the presentation comes from **Adapters.py** 'get_results' from the adapter attribute in the **Steps.py** file. The used libraries was **TKinter** and **matplotlib**, but vary acordilly with the execution flow necessities.

Code 4 illustrates a preview class and a table presenter class.

```

1
2 import copy
3 import Tkinter
4 import numpy as np
5 from typing import Dict, Tuple
6 from matplotlib.figure import Figure
7 from ..GUI.Widgets import CanvasTable
8 from ..GUI.Presenters import Presenter, PlotbookPresenter
9
10
11 class CloudSimPresenter(Presenter):
12
13     def __init__(self):
14         super(CloudSimPresenter, self).__init__()
15
16         self.table = None
17
18         self.canvas_frame = None

```

```

19
20     def present_results(self, frame, results, options=None):
21         return 0
22
23     def present_partials(self, frame, step_results, options=
24         ↪ None):
25         raise NotImplementedError
26
27     def get_info(self, step_results, options=None):
28
29         simulation_preview = (
30             "Time: {} hours\n"
31             "Price: {} USD/h\n"
32             ).format('%%.2e' % step_results['original_time'], '%.2
33             ↪ e' % round(step_results['original_price'], 2)
34         )
35         return simulation_preview
36
37 class NSGATablePresenter(Presenter):
38
39     def __init__(self):
40
41         super(NSGATablePresenter, self).__init__()
42
43         self.canvas = None
44
45         self.og_table = None
46
47         self.sol_table = None
48
49     def present_partials(self, frame, step_results, options=
50         ↪ None):
51         raise NotImplementedError
52
53     def present_results(self, frame, results, options=None):
54
55         if 'nsga_solutions' not in results['dsdse']:
56             return 0
57
58         cell_height = 25
59
60         table_options = {
61             'pos': (2, 3*(cell_height+2)),
62             'cells_width': [250, 250, 250],
63             'font_height': 12,
64             'cell_height': cell_height,
65             'nbr_of_columns': 3,
66             'nbr_of_rows': 6,
67             'center': False,
68         }

```

```

67     self.canvas = Tkinter.Canvas(frame)
68
69     solutions = results['dsdse']['nsga_solutions']
70
71     sorted_solution = results['dsdse']['sorted_nsga']
72
73     nbr_of_solutions = len(sorted_solution)
74
75     height = table_options['cell_height'] * (6 +
76         ↳ nbr_of_solutions + 6)
77
78     self.canvas.config(width=options['width'], height=
79         ↳ height)
80
81     self.canvas.pack(
82         fill=Tkinter.BOTH,
83         expand=True
84     )
85
86     self.canvas.create_text(2, 2*(cell_height+2), text="
87         ↳ NSGA-II Generated Architectures",
88         anchor=Tkinter.NW)
89
90     table_options['cells_width'] = [445, 145, 140, 140]
91
92     table_options['nbr_of_rows'] = nbr_of_solutions + 1
93
94     table_options['nbr_of_columns'] = 3
95
96     solutions_data = [
97         ['Architecture', 'Predicted cost (USD/h)', '
98             ↳ Predicted time (hours)'],
99     ]
100
101     for s in sorted_solution:
102         solutions_data.append([
103             s,
104             '%.2e' % round(solutions[s]['cost_pred'], 10)
105             ↳ ,
106             '%.2e' % round(float(solutions[s]['time_pred']
107                 ↳ ), 10),
108         ])
109
110     table_options['data'] = solutions_data
111
112     self.sol_table = CanvasTable(self.canvas,
113         ↳ table_options)
114
115     return height

```

```

111
112     def get_info(self, step_results, options=None):
113         raise NotImplemented

```

Listing 4: Presenters Python code

2.5 MultiExplorer{flow_name}

The **MultiExplorer{flow_name}.py** file is the main file of an execution flow. It contains the execution flow class, which will be included in the MultiExplorer execution flow list and displayed as an option. This class includes the execution flow description (get.info), the steps of the flow (self.steps), the name to be displayed in the MultiExplorer interface (get.label), manages the folder containing each project, and handles the execution of the flow.

Code 5 illustrates the VM execution flow.

```

1
2 import os
3 import tkinter as tk
4 from ..config import PATH_RUNDIR
5 from ..Infrastructure.Events import Event
6 from Steps import CloudSimStep, NSGAIIDSEStep
7 from ..Infrastructure.ExecutionFlow import ExecutionFlow
8 from Presenters import BruteForceTablePresenter,
9     ↳ NSGATablePresenter, CloudSimPresenter, NSGAPresenter,
10     ↳ BruteForcePresenter
11
12 class MultiExplorerVMExecutionFlow(ExecutionFlow):
13
14     @staticmethod
15     def get_info():
16         return (
17             "This workflow enables the configuration of
18             ↳ virtual machines based on user requirements
19             ↳ and application constraints. \n"
20             + "MultiExplorer-VM utilizes CloudSim as the
21             ↳ cloud simulation platform. \n"
22             + "The design space exploration (DSE) is
23             ↳ conducted using an NSGA2-based algorithm. \n
24             ↳ "
25             + "In the current version, MultiExplorer-VM also
26             ↳ incorporates a brute-force algorithm to
27             ↳ explore all viable design alternatives. \n"
28             + "The brute-force approach serves as a
29             ↳ validation step for our DSE methodology."
30         )
31
32     def __new__(cls):
33         if not hasattr(cls, 'instance'):
34             cls.instance = super(
35                 MultiExplorerVMExecutionFlow,

```

```

26         cls
27     ).__new__(cls)
28
29     return cls.instance
30
31 def __init__(self):
32     super(MultiExplorerVMExecutionFlow, self).__init__()
33
34     self.steps = [
35         CloudSimStep(),
36         NSGAIIDSEStep()
37     ]
38
39 @staticmethod
40 def get_label():
41     return 'MultiExplorer Virtual Machines'
42
43 def get_output_path(self):
44     return (
45         PATH_RUNDIR
46         + "/" + MultiExplorerVMExecutionFlow.
47             ↪ get_label().replace(' ', '_')
48     )
49
50 def setup_dirs(self):
51     output_path = self.get_output_path()
52
53     if not os.path.exists(output_path):
54         os.makedirs(output_path)
55
56     nbr_of_dirs = len(next(os.walk(output_path))[1])
57
58     nbr_of_dirs = nbr_of_dirs & 63
59
60     output_path = output_path + "/" + "{:02d}".format(
61         ↪ nbr_of_dirs)
62
63     if not os.path.exists(output_path):
64         os.makedirs(output_path)
65
66     for step in self.steps:
67         step.set_output_path(output_path)
68
69 def execute(self):
70     self.setup_dirs()
71
72     ExecutionFlow.execute(self)
73
74 def get_results(self):
75     return {
76         "cloudsim": self.steps[0].get_results(),

```

```

75         "dsdse": self.steps[1].get_results()
76     }
77
78     def get_presenters(self):
79         return [
80             NSGATablePresenter(),
81             NSGAPresenter(),
82             BruteForceTablePresenter(),
83             BruteForcePresenter(),
84             CloudSimPresenter()
85         ]
86
87     def handle_step_failure(self, step):
88         tkMessageBox.showerror(
89             "Execution Failure",
90             "The " + step.get_label() + " Step execution wasn
           ↳ 't successful. " + str(step.
           ↳ execution_exception)
91         )
92
93         self.fire(Event.FLOW_EXECUTION_FAILED)
94
95     def finish(self):
96         self.fire(Event.FLOW_EXECUTION_ENDED)

```

Listing 5: Virtual machines execution flow Python code

2.6 How to include a execution flow in MultiExplorer list

To include an execution flow in MultiExplorer, the flow must be registered in the */multiexplorer/MultiExplorer/src/Infrastructure/Registries.py* file.

To register a flow, the 'register_flow_class' function should be called with the execution flow class and its corresponding name.

```

1
2 class ExecutionFlowRegistry(object):
3
4     ...
5
6     def __init__(self):
7         self.flow_classes = {} # type: Dict[str, 'class']
8
9         self.register_flow_class(
10             CPUHeterogeneousMulticoreExplorationExecutionFlow
           ↳ ,
11             CPUHeterogeneousMulticoreExplorationExecutionFlow
           ↳ .get_label()
12         )
13
14         self.register_flow_class(
15             MultiExplorerVMEExecutionFlow,
16             MultiExplorerVMEExecutionFlow.get_label()

```

```
17         )
18
19     self.register_flow_class(
20         MultiexplorerGPGPUExecutionFlow,
21         MultiexplorerGPGPUExecutionFlow.get_label()
22     )
23
24     ...
```

Listing 6: Register an execution flow Python code